Dynamic Questionnaire Analysis

Programming Assignment

Introduction

In many applications (surveys, medical assessments, financial applications), questionnaires have conditional logic where certain questions only appear based on previous answers. For example, if someone answers “Yes” to “Do you own a car?”, they might see follow-up questions about the car's make and model. This creates a branching structure where different answer paths lead to different sets of questions.

Every problem in this assignment must be solved with recursion. Each function is expected to walk the question graph through recursive calls (a recursive helper function is fine); iterative loops may be used only for bookkeeping such as building an index, iterating over a question's answers, or sorting the final result.

Dataset

You are provided with questionnaire data in JSON format. Each question has a unique ID, question text, and a question type. Questions of type `radio` or `select` carry a list of possible answers, each with its own `next_question_id`; questions of type `text` or `checkbox` have a single unconditional `next_question_id`. A `next_question_id` of `null` ends the questionnaire.

TypeScript Interfaces

```
interface Answer {
  id: string;
  text: string;
  next_question_id: string | null;   // null means end of questionnaire
}

interface Question {
  id: string;
  text: string;
  type: "radio" | "select" | "text" | "checkbox";
  answers?: Answer[];                // Only for radio/select types
  next_question_id?: string | null;  // For text/checkbox (unconditional next)
}

interface Questionnaire {
  questions: Question[];
  start_question_id: string;
}
```

JSON Structure

```
{
  "start_question_id": "q1",
  "questions": [
    {
      "id": "q1", "text": "Do you own a car?", "type": "radio",
      "answers": [
        { "id": "a1_yes", "text": "Yes", "next_question_id": "q2" },
        { "id": "a1_no",  "text": "No",  "next_question_id": "q5" }
      ]
    },
    {
      "id": "q2", "text": "What type of car do you own?", "type": "select",
      "answers": [
        { "id": "a2_sedan",  "text": "Sedan",      "next_question_id": "q3" },
        { "id": "a2_suv",    "text": "SUV",        "next_question_id": "q3" },
        { "id": "a2_truck",  "text": "Truck",      "next_question_id": "q4" },
        { "id": "a2_sports", "text": "Sports Car", "next_question_id": "q4" }
      ]
    },
    {
      "id": "q3", "text": "How many passengers can your vehicle seat?", "type": "radio",
      "answers": [
        { "id": "a3_2to5",  "text": "2-5", "next_question_id": "q6" },
        { "id": "a3_6plus", "text": "6+",  "next_question_id": "q6" }
      ]
    },
    {
      "id": "q4", "text": "Do you use your vehicle for work purposes?", "type": "radio",
      "answers": [
        { "id": "a4_yes", "text": "Yes", "next_question_id": "q7" },
        { "id": "a4_no",  "text": "No",  "next_question_id": "q6" }
      ]
    },
    {
      "id": "q5", "text": "What is your primary mode of transportation?", "type": "select",
      "answers": [
        { "id": "a5_public", "text": "Public Transit", "next_question_id": "q6" },
        { "id": "a5_bike",   "text": "Bicycle",        "next_question_id": "q6" },
        { "id": "a5_walk",   "text": "Walking",        "next_question_id": "q6" }
      ]
    },
    {
      "id": "q6", "text": "What is your annual commute distance?",
      "type": "text", "next_question_id": "q8"
    },
    {
      "id": "q7", "text": "What percentage of your vehicle use is for work?",
      "type": "text", "next_question_id": "q8"
    },
    {
      "id": "q8", "text": "Are you satisfied with your current transportation situation?",
"type": "radio",
      "answers": [
        { "id": "a8_yes", "text": "Yes", "next_question_id": null },
        { "id": "a8_no",  "text": "No",  "next_question_id": "q9" }
      ]
    },
    {
      "id": "q9", "text": "What would you like to improve?",
      "type": "text", "next_question_id": null
    }
```

```
  ]
}
```

Questionnaire Flow Visualization

```
q1: Own a car?
  Yes -> q2: Car type?
         Sedan  -> q3: Passengers?  (2-5 | 6+)  -> q6 -> q8
         SUV    -> q3: Passengers?  (2-5 | 6+)  -> q6 -> q8
         Truck  -> q4: Work use?    Yes -> q7 -> q8
                                    No  -> q6 -> q8
         Sports -> q4: Work use?    Yes -> q7 -> q8
                                    No  -> q6 -> q8
  No  -> q5: Primary transport?
         Public Transit | Bicycle | Walking -> q6 -> q8

q6: Commute distance? -> q8
q7: Work %?           -> q8
q8: Satisfied?        Yes -> END
                      No  -> q9: Improve? -> END
```

> Note that the graph is a DAG, not a tree: q6, q8 and q9 are each reachable from several different branches. Your recursion must therefore handle a question being visited through more than one path.

Problem 1: Find Longest Question Path

Description

Determine the longest possible path through the questionnaire—that is, the maximum number of questions a user could answer before reaching the end. This represents the most detailed response path.

Function Signature

```
interface LongestPath {
  length: number;
  question_ids: string[];
  answer_path: string[];  // Answer IDs chosen along this path
}

function findLongestQuestionPath(questionnaire: Questionnaire): LongestPath {
  /**
   * Find the longest possible path through the questionnaire.
   *
   * @param questionnaire - The questionnaire object with questions and start point
   * @returns Object containing the length, question IDs, and answer choices for
   *          the longest path
   */
  // Your implementation here
}
```

Input Format

- `questionnaire`: An object containing the array of questions and the starting question ID

Output Format

Return an object containing:

- `length`: The number of questions in the longest path — this must equal `question_ids.length`
- `question_ids`: Array of question IDs, in order, for this path
- `answer_path`: Array of answer IDs chosen along this path. Only `radio` and `select` questions contribute an entry; `text` and `checkbox` questions have no answer choice, so `answer_path` is normally shorter than `question_ids`

Several distinct paths may share the maximum length. Return any one of them; a test should assert on `length` and on the validity of the returned path, not on one specific branch.

Example

```
findLongestQuestionPath(questionnaire);  // Returns (one valid answer):
{
  length: 6,
  question_ids: ["q1", "q2", "q4", "q7", "q8", "q9"],
  answer_path:  ["a1_yes", "a2_truck", "a4_yes", "a8_no"]
}
// Path: Own car (Yes) -> Truck -> Work use (Yes) -> Work % -> Satisfied (No) -> Improve

// ["q1", "q2", "q3", "q6", "q8", "q9"] is also of length 6 and equally correct.
```

Requirements

- Use **recursion** to explore the possible paths: the longest path from a question is that question plus the longest of the paths from each of its next questions
- Handle questions with multiple possible answers, and questions with a single unconditional next question
- Track the answer IDs that lead to the maximum-length path
- Guard against cycles: keep the set of questions on the current path and do not re-enter one
- Plain recursion re-explores shared sub-paths, so its cost grows with the number of distinct paths (22 for this dataset). Caching the best result per question id is optional; it brings the cost down to O(n × m), where n is the number of questions and m the average number of answers per question

---

Problem 2: Find Shortest Question Path

Description

Determine the shortest possible path through the questionnaire—the minimum number of questions a user must answer to complete the survey. This represents the quickest completion route.

Function Signature

```
interface ShortestPath {
  length: number;
  question_ids: string[];
  answer_path: string[];
}

function findShortestQuestionPath(questionnaire: Questionnaire): ShortestPath {
  /**
   * Find the shortest possible path through the questionnaire.
   *
   * @param questionnaire - The questionnaire object
   * @returns Object containing the length, question IDs, and answer choices for
   *          the shortest path
   */
  // Your implementation here
}
```

Input Format

- `questionnaire`: An object containing the array of questions and the starting question ID

Output Format

Same shape as Problem 1: `length` must equal `question_ids.length`, and `answer_path` lists only the answer IDs of `radio`/`select` questions along the path.

Example

```
findShortestQuestionPath(questionnaire);  // Returns (one valid answer):
{
  length: 4,
  question_ids: ["q1", "q5", "q6", "q8"],
  answer_path:  ["a1_no", "a5_public", "a8_yes"]
}
// Path: Own car (No) -> Public Transit -> Commute distance -> Satisfied (Yes) -> END
// "a5_bike" and "a5_walk" lead to the same question sequence and are equally correct.

// For comparison, the "Yes" branch is longer:
// ["q1", "q2", "q3", "q6", "q8"] has length 5, so it is not the shortest path.
```

Requirements

- Use **recursion**: the shortest path from a question is that question plus the shortest of the paths from each of its next questions
- Consider all possible answer branches — the shortest route is not necessarily the first branch listed
- Return any valid shortest path if several exist with the same length
- Guard against cycles, as in Problem 1

Problem 3: Count All Possible Paths

Description

Calculate the total number of distinct paths through the questionnaire. Each unique *combination of answer choices* that leads from the start question to the end counts as a separate path. Two routes that visit the same questions but differ in any answer choice are therefore different paths — for example, answering q3 with “2-5” and answering it with “6+” both continue to q6, yet they count as two paths.

Function Signature

```
function countAllPossiblePaths(questionnaire: Questionnaire): number {
  /**
   * Count the total number of distinct paths through the questionnaire.
   *
   * @param questionnaire - The questionnaire object
   * @returns The total number of distinct answer combinations from start to end,
   *          or -1 if the questionnaire contains a cycle (infinitely many paths)
   */
  // Your implementation here
}
```

Input Format

- `questionnaire`: An object containing the array of questions and the starting question ID

Output Format

Return an integer representing the total number of distinct paths, or `-1` if a cycle is detected.

Example

```
countAllPossiblePaths(questionnaire);
// Returns: 22

// Worked out bottom-up, where paths(q) is the number of ways to finish from q:
//   paths(q9) = 1
//   paths(q8) = 1 (Yes -> END) + paths(q9)            = 2
//   paths(q7) = paths(q8)                             = 2
//   paths(q6) = paths(q8)                             = 2
//   paths(q5) = 3 x paths(q6)                         = 6
//   paths(q4) = paths(q7) + paths(q6)                 = 4
//   paths(q3) = 2 x paths(q6)                         = 4
//   paths(q2) = paths(q3) + paths(q3) + paths(q4) + paths(q4) = 16
//   paths(q1) = paths(q2) + paths(q5)                 = 22
```

Requirements

- Use **recursion** to traverse all branches: the count for a question is the sum of the counts of its next questions, with a `next_question_id` of `null` counting as exactly one completed path
- Count each answer choice as its own branch, even when two answers lead to the same next question
- Detect cycles and return `-1` when the number of paths would be infinite

---

Problem 4: Find All Questions Reachable from an Answer

Description

Given a specific question and a specific answer to that question, find all questions that could possibly be reached later in the questionnaire by following that answer. This shows the downstream impact of each answer choice.

Function Signature

```
interface ReachableQuestion {
  question_id: string;
  question_text: string;
  min_depth: number;  // Minimum steps from the starting answer to reach this question
}

function findReachableQuestions(
  questionnaire: Questionnaire,
  questionId: string,
  answerId: string
): ReachableQuestion[] {
  /**
   * Find all questions reachable after selecting a specific answer.
   *
   * @param questionnaire - The questionnaire object
   * @param questionId    - The ID of the question being answered
   * @param answerId      - The ID of the answer choice
   * @returns Array of reachable questions with their minimum depth
   */
  // Your implementation here
}
```

Input Format

- `questionnaire`: The questionnaire object
- `questionId`: ID of the starting question
- `answerId`: ID of the specific answer choice on that question

Output Format

Return an array of objects, each containing:

- `question_id`: The ID of a reachable question
- `question_text`: The text of that question
- `min_depth`: The minimum number of steps from the starting answer to that question; the question the answer leads to directly has `min_depth: 1`

Sort by `min_depth` ascending, then by `question_id` ascending. Each reachable question appears exactly once, even if several routes lead to it.

Example

```
findReachableQuestions(questionnaire, "q2", "a2_truck");  // Returns:
[
  { question_id: "q4", question_text: "Do you use your vehicle for work purposes?",
min_depth: 1 },
  { question_id: "q6", question_text: "What is your annual commute distance?",
min_depth: 2 },
  { question_id: "q7", question_text: "What percentage of your vehicle use is for work?",
min_depth: 2 },
  { question_id: "q8", question_text: "Are you satisfied with your current transportation
situation?", min_depth: 3 },
  { question_id: "q9", question_text: "What would you like to improve?",
min_depth: 4 }
]

findReachableQuestions(questionnaire, "q8", "a8_yes");   // Returns: [] (answer ends the
questionnaire)
findReachableQuestions(questionnaire, "q2", "a9_nope");  // Returns: [] (unknown question or
answer ID)
```

Requirements

- Use **recursion** with depth tracking: recurse into each next question with `depth + 1`
- Record the *minimum* depth for each reachable question: when a question is reached again at a smaller depth, update it and recurse through it once more; when it is reached at an equal or greater depth, stop. This also terminates the recursion if the data ever contains a cycle
- Return an empty array if the answer leads directly to the end, or if the question ID or answer ID does not exist

---

Problem 5: Calculate Question Appearance Probability

Description

Calculate the probability that each question will be shown to a user, assuming all answer choices at a question are equally likely to be selected. This helps identify which questions are most or least frequently seen and can inform questionnaire design.

Function Signature

```
interface QuestionProbability {
  question_id: string;
  question_text: string;
  probability: number;         // Value between 0 and 1
  expected_position: number;   // Average position in the questionnaire, given the
                               // question is shown (first question = 1)
}

function calculateQuestionProbabilities(
  questionnaire: Questionnaire
): QuestionProbability[] {
  /**
   * Calculate the probability of each question being shown.
   *
   * @param questionnaire - The questionnaire object
   * @returns Array of questions with their appearance probabilities
   */
  // Your implementation here
}
```

Input Format

- `questionnaire`: The questionnaire object

Output Format

Return an array of objects, each containing:

- `question_id`: The question ID
- `question_text`: The question text
- `probability`: Probability that this question appears (0.0 to 1.0)
- `expected_position`: Average position at which it appears, *conditional on it appearing* (first question = 1)

Sort by `probability` descending, then by `question_id` ascending. Questions that are unreachable from the start question have probability 0 and are reported with `expected_position: 0`.

Expected Result for the Dataset

A question with a single unconditional next question passes its whole probability on; a question with *k* answers passes `probability / k` along each answer. Walking the flow from q1 gives:

question_id	probability	expected_position	why
q1	1.0	1.0	start question
q8	1.0	4.5	every path reaches it: position 4 half the time, 5 half the time
q6	0.875	3.4286	0.5 from q5 + 0.25 from q3 + 0.125 from q4 (No)
q2	0.5	2.0	q1 = Yes
q5	0.5	2.0	q1 = No
q9	0.5	5.5	q8 = No, one step after q8
q3	0.25	3.0	0.5 x 2/4 (Sedan or SUV)

q4	0.25	3.0	0.5 x 2/4 (Truck or Sports Car)
q7	0.125	4.0	0.25 x 1/2 (q4 = Yes)

As a self-check, the probabilities sum to 5.0, which is the expected number of questions a user answers.

Example

```
calculateQuestionProbabilities(questionnaire);  // Returns:
[
  { question_id: "q1", question_text: "Do you own a car?",
    probability: 1.0,   expected_position: 1.0 },
  { question_id: "q8", question_text: "Are you satisfied with your current transportation
situation?",
    probability: 1.0,   expected_position: 4.5 },
  { question_id: "q6", question_text: "What is your annual commute distance?",
    probability: 0.875, expected_position: 3.4286 },
  { question_id: "q2", question_text: "What type of car do you own?",
    probability: 0.5,   expected_position: 2.0 },
  // ... q5, q9, q3, q4, q7
]
```

Requirements

- Assume all answer choices at a given question are equally likely
- Use **recursion** to propagate probability (and position) forward from the start question; a question reached through several branches accumulates the probability of each
- The first question always has probability 1.0 and expected position 1.0
- Handle conditional branching correctly: a `text`/`checkbox` question forwards all of its probability to its single next question
- Round `probability` and `expected_position` to 4 decimal places
- Because the flow is a DAG and not a tree, process a question only once its incoming branches have all been accounted for, or accumulate contributions into a map keyed by question id before reporting